📄 **Technical Specification Document**

Project: AI Desktop Assistant (CTRL AI)
Version: 1.0
Author: Y. Tejas
Date: 2025-10-24

---

1. Introduction

1.1 Purpose

The AI Desktop Assistant is a local-first, AI-driven application designed to control and manage a desktop environment through **text and voice commands**. It allows users to **search, open, edit, organize files, control applications, manage browser tabs, and integrate with emails/messages**, while maintaining strict **security, privacy, and audit controls**.

The assistant combines **natural language understanding (LLM)** with **modular automation skills** to provide productivity-enhancing desktop operations.

1.2 Goals

- Enable **natural interaction** (text + optional voice) with the desktop.
- Provide **modular skills** (files, apps, browser, system, email).
- Maintain **user trust** via permissions, previews, undo, and audit logs.
- Be **cross-platform long-term**, but start with **Windows 11 MVP**.

1.3 Non-Goals

- Full GUI automation of arbitrary third-party apps (too brittle).
- Cloud-only operation (must support local/offline mode).
- Autonomous execution without human approval (dangerous).

---

2. System Overview

2.1 High-Level Architecture

```
+---------------------------+
|       User (Voice/Text)  |
+-------------+-------------+
            |
       [Command Input]
            |
+-------------v----------------------------+
|           Orchestrator                  |
|  - Intent Parsing (LLM/Regex)           |
|  - Schema Validator (JSON intents)      |
|  - Permission Checker                   |
|  - Execution Router                     |
+-------------+----------------------------+
            |
       [Skill Invocation]
            |
+------+------+-----+------+----------------+
| File Skill | App Skill | Browser Skill    |
| Email Skill| System    | Custom Plugins   |
+------------+----------+-------------------+
            |
       [OS APIs / Extensions]
            |
      +---------------------+
      | Windows APIs / OS   |
      | File System, Apps,  |
      | PowerShell, UIA,    |
      | Chrome Extension    |
      +---------------------+
```

2.2 Key Components

- **UI Shell**: Tray icon + command palette (hotkey).
- **Orchestrator**: Core engine routing user intent → skills.
- **LLM NLU**: Converts free text/voice into structured JSON intents.
- **Schema Validator**: Enforces strict JSON schema before execution.
- **Skills**: Modular capability modules (file, app, browser, etc.).
- **Permission Layer**: Defines allowed paths, apps, and operations.
- **Audit & Undo Engine**: Logs all actions, supports rollback.
- **Voice Engine (optional)**: Whisper ASR + TTS response.

---

3. Functional Requirements

3.1 File Skill

- Search files (by name, extension, recency).
- Open files with default app.
- Organize by type/date into folders.
- Rename/move files.
- Delete (safe → recycle bin, confirm required).

3.2 App Skill

- Launch applications (mapped by name → executable).
- Quit applications.
- Switch focus between running apps.

3.3 Browser Skill (via Extension + Native Messaging)

- List tabs.
- Switch to a tab by title/domain.
- Open new tab.
- Close tab.

3.4 Email/Message Skill (future phase)

- Connect via OAuth.
- List unread emails/messages.
- Compose and send (with confirmation).

3.5 System Skill

- Adjust volume/brightness.
- Toggle WiFi/Bluetooth (Windows APIs).
- Show system info (battery, CPU).

3.6 Audit & Permissions

- Maintain an **audit log** of all commands.
- Support **undo** where feasible.
- Enforce **per-skill allowlists** (folders, apps, operations).

---

4. Non-Functional Requirements

- **Security-first**: no action without permission, preview, or logging.
- **Performance**: file search < 200 ms on indexed folders.
- **Reliability**: transactional batch ops with rollback.
- **Portability**: modular backend; initial target Windows 11.
- **Extensibility**: plugins for new skills.

---

5. Security & Privacy

5.1 Permission JSON Schema

```
{
  "skill": "files",
  "allowed_paths": ["C:/Users/Tejas/Desktop"],
  "allowed_ops": ["read", "open", "move"],
  "danger_ops": ["delete", "overwrite"],
  "danger_ops_require": "explicit_confirmation"
}
```

5.2 LLM Output JSON Schema

```
{
  "intent": "files.move",
  "args": {
    "query": "type:png on Desktop older_than:30d",
    "destination": "C:/Users/Tejas/Pictures/Screenshots",
```

```
    "simulate": true
  }
}
```

5.3 Threat Model

- **Prompt Injection**: Mitigation → schema validation + deny unsafe intents.
- **Privilege Escalation**: Mitigation → least privilege, UAC only when required.
- **Exfiltration Risk**: Mitigation → block outbound transfer unless explicit.
- **Plugin Abuse**: Mitigation → signed plugins, sandboxing.

---

6. Technical Stack

6.1 Language & Frameworks

- **Backend**: Python 3.12
- **Packaging**: PyInstaller
- **UI**: PyQt5 or Tauri (future)
- **Automation**: pywinauto, uiautomation, PowerShell APIs
- **File Indexing**: sqlite3 with FTS5 + watchdog
- **Voice Recognition**: faster-whisper
- **LLM**: OpenAI GPT or Local Llama 3.1 (JSON output enforced)
- **Browser Control**: Chrome/Edge Extension + Native Messaging Host

6.2 Data Storage

- SQLite database for:
    - File index (path, name, extension, mtime).
    - Permissions.
    - Audit logs.

---

7. Development Roadmap

Phase 1 (Week 1–2) — MVP Core

- Command palette (hotkey-based).
- File skill: search, open, organize.
- SQLite-based indexer.
- Audit log + undo for file ops.

Phase 2 (Week 3–4) — App & Browser

- App skill: launch/quit/focus.
- Browser skill via extension.
- Permission manager UI.

Phase 3 (Week 5–6) — Voice + System

- Push-to-talk voice input (Whisper).
- System skill (volume, brightness).
- JSON schema validator + LLM integration.

Phase 4 (Week 7–8) — Email & Messages

- Gmail/Outlook connectors.
- Send/read emails with OAuth.
- Extended audit & rollback.

---

8. Testing & QA

8.1 Unit Tests

- File operations (simulate vs execute).
- Schema validation (invalid → refuse).

8.2 Integration Tests

- File indexing + search.
- Command routing to correct skills.

8.3 Security Tests

- Red-team prompt injection attempts.
- Fuzz invalid JSON outputs.
- Attempt unauthorized access (blocked).

8.4 User Testing

- Simulate “organize Desktop” scenario.
- Verify undo, logs, and dry-run preview.

---

9. Deployment

- Distributed as a **signed installer (MSI/EXE)**.
- Auto-update via secure channel (HTTPS + signed binaries).

- Local-only mode (no cloud calls).
- Uninstaller must remove watchers, index, and configs.

---

10. Risks & Mitigations

Risk	Mitigation
User distrust (too much access)	Transparent logs, simulation, granular permissions
LLM hallucinations	Strict JSON schema validator
High CPU from indexing	Throttled indexer, exclude rules
Cross-platform complexity	Target Windows-only for MVP
Browser automation brittle	Use extension + native messaging